



ITI · AI Agents Development using Python track

Python Programming for AI Applications

Eng. Fatma Elraey*Day 4 — Classes, Inheritance & Polymorphism (Object-Oriented Python)*

Field	Detail
Instructor	Fatma Elraey
Track	Python for AI Applications (6-Day Course)
Session	Day 4 of 6
Focus	Structuring code around classes and objects — constructors, inheritance, polymorphism, and encapsulation, the foundation beneath every scikit-learn estimator and PyTorch module.

Quick Recap — Day 3

On Day 3 you organized code into modules and packages, protected your programs with exception handling, and read and wrote the files every dataset ultimately lives in. Today we go one level deeper: instead of just grouping functions together, we bundle data and behavior into a single unit — a class — and learn how classes can build on one another through inheritance, present many faces through polymorphism, and hide their internals through encapsulation.

Day 4 Outline

1. From Procedural to Object-Oriented Python
2. Classes, Constructors & Variables
3. Instance, Class & Static Methods
4. Inheritance & Multiple Inheritance
5. Polymorphism & Encapsulation
6. Special Methods & Functional Tips

01

From Procedural to Object-Oriented Python

Every Python program you have written so far sits somewhere on a ladder of organization. Plain scripts with jumps and repeated logic sit at the bottom; functions let you reuse blocks of logic; modules let you group related functions into files. Object-oriented programming (OOP) is the next rung: it bundles data and the functions that operate on that data into a single unit called an object.

From tangled scripts to structured, AI-ready code

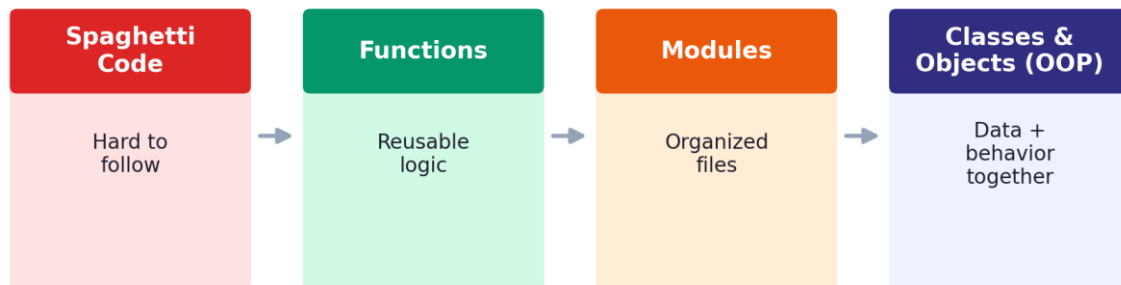


Figure 4.1 — Four levels of code organization, from tangled scripts to full OOP.

An object has properties (its data, also called attributes) and methods (the functions it can perform). A trained model object, for example, might carry a name and an accuracy score as properties, and expose `train()` and `predict()` as methods.

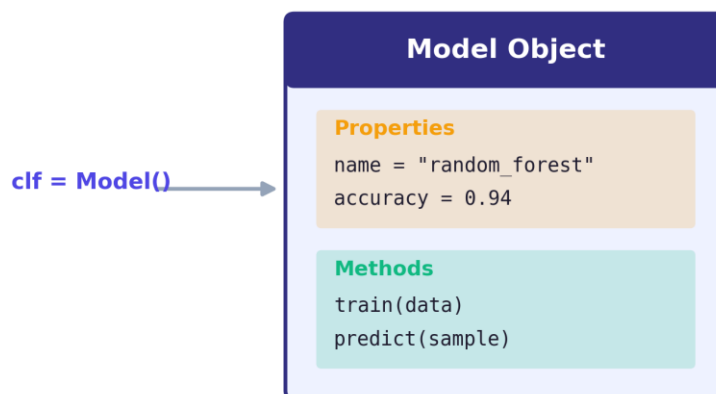


Figure 4.2 — An object bundles data (properties) and behavior (methods) together.

A class is the template that describes what properties and methods its objects will have. Every object you create from a class is called an instance of that class.

```
class Model:
    pass

clf = Model()
print(type(clf))
# <class '__main__.Model'>
```

AI CONNECTION Every scikit-learn estimator you call — `LinearRegression()`, `RandomForestClassifier()`, `StandardScaler()` — is an instance of a class. Calling `.fit()` or `.predict()` on it is exactly the method-calling pattern you are about to learn.

✓ QUICK CHECK — From Procedural to OOP

Q1. What two things does an object bundle together?

Answer: Its data (properties/attributes) and the functions that act on that data (methods).

Q2. What is the relationship between a class and an object?

Answer: A class is the template; an object (instance) is a concrete thing built from that template.

02 Classes, Constructors & Variables

Defining a class only needs the keyword `class` and a name. An empty class body needs the placeholder `pass`:

```
class Human:
    pass
```

Calling the class like a function creates an object — an instance of that class:

```
man = Human()
print(man)
# <__main__.Human object at 0x000...>
```

The constructor is a special method, `__init__`, that Python calls automatically the moment an object is created. It is where you set up whatever the object needs to start with:

```
class Human:
    def __init__(self):
        print("Hi there")

man = Human()
# Hi there
```

`self` refers to the specific object being created or used — it is always the first parameter of an instance method. Passing arguments into `__init__` lets each object start with its own values, called instance variables:

```
class Human:
    def __init__(self, name):
        self.name = name

man = Human("Ahmed")
print(man.name)
# Ahmed
```

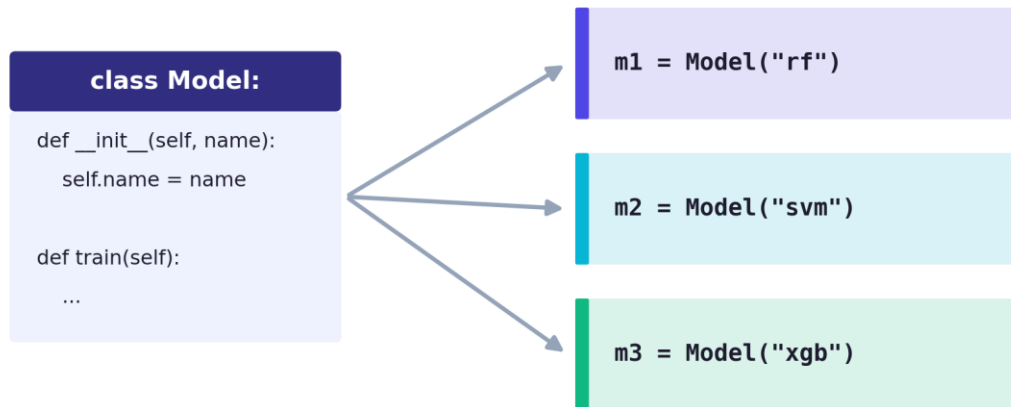


Figure 4.3 — One class definition, many independent instances, each with its own data.

A class variable, on the other hand, is defined directly inside the class body (not inside `__init__`) and is shared by every instance unless an instance overrides it on itself:

```
class Human:
    faults = 0 # class variable

    def __init__(self, name):
        self.name = name

man = Human("Ahmed")
man2 = Human("Mohamed")
man.faults = 1 # only affects man
print(man.faults, man2.faults, Human.faults)
# 1 0 0
Human.faults = 2 # changes the shared value
print(man.faults, man2.faults, Human.faults)
# 1 2 2
```

AI CONNECTION Instance variables are exactly what holds a fitted model's learned weights (`model.coef_`) — every estimator you train has its own copy. Class variables show up as shared defaults, such as a counter of how many pipeline runs have executed across every object of a logging class.

✓ QUICK CHECK — Classes, Constructors & Variables

Q1. When does `__init__` run, and what is `self`?

Answer: `__init__` runs automatically the moment an object is created; `self` is a reference to that specific object.

Q2. How does a class variable differ from an instance variable?

Answer: A class variable is defined in the class body and shared by all instances by default; an instance variable is set on `self` inside `__init__` (or later) and belongs to just that object.

Q3. If `man.faults = 1` is set on one instance, does it change `Human.faults`?

Answer: No — assigning to `man.faults` creates a new instance attribute on `man` only; the shared class variable `Human.faults` is untouched.

03 Instance, Class & Static Methods

An instance method is a function defined inside a class that takes `self` as its first parameter, so it can read and change that specific object's data:

```
class Human:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print("My name is " + self.name)

man = Human("Ahmed")
man.speak()
# My name is Ahmed
```

A class method takes `cls` instead of `self`, and operates on the class itself rather than one instance — useful for state that belongs to the whole class, like a running count:

```
class Human:
    population = 0

    def __init__(self, name):
        self.name = name
        Human.population += 1

    @classmethod
    def total_created(cls):
        return cls.population

Human("Ahmed")
Human("Mohamed")
print(Human.total_created())
# 2
```

A static method takes neither `self` nor `cls` — it is a plain helper function that logically belongs to the class, grouped there for organization:

```
class Human:
    @staticmethod
    def is_adult(age):
        return age >= 18

print(Human.is_adult(20))
# True
```

Class Method vs. Static Method

```
# Class Method — implicitly receives cls (the class itself).
# Use it when the logic needs to read or change class-level state.

# Static Method — receives neither self nor cls.
# Use it for a helper function that is merely related to the class.
```

AI CONNECTION *scikit-learn's `StandardScaler.fit()` is an instance method — it stores the mean and variance on `self`. A dataset-loading class might use a `@classmethod` like `from_csv(cls, path)` as an alternative constructor, and a `@staticmethod` like `is_valid_label(y)` as a pure helper.*

✓ QUICK CHECK — Instance, Class & Static Methods

Q1. What is the first parameter of an instance method, and what does it give you?

Answer: *self* — a reference to the specific object the method was called on, so you can read or update its data.

Q2. Why would you reach for a `@classmethod` instead of a regular method?

Answer: When the logic needs to operate on the class as a whole (shared state, or building an alternative constructor) rather than one instance.

Q3. What distinguishes a `@staticmethod` from a `@classmethod`?

Answer: A static method receives neither *self* nor *cls* — it behaves like a plain function that is simply grouped inside the class for organization.

04 Inheritance & Multiple Inheritance

Inheritance lets one class (the child) reuse and extend the properties and methods of another class (the parent), instead of rewriting them from scratch.

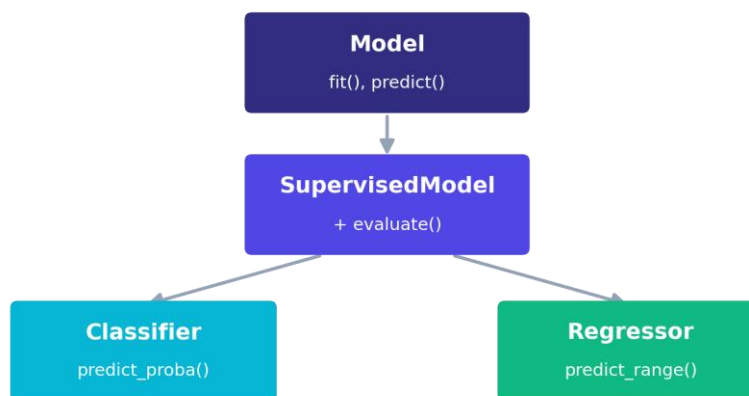


Figure 4.4 — A base class's methods flow down to every subclass beneath it.

```
class Model:
    def __init__(self, name):
        self.name = name

    def describe(self):
        print("Model: " + self.name)

class Classifier(Model):
    def __init__(self, name, n_classes):
        super().__init__(name)
        self.n_classes = n_classes

clf = Classifier("tree", 3)
clf.describe() # inherited from Model
# Model: tree
```

`super()` gives you access to the parent class from inside the child, so `Classifier` does not have to re-implement `__init__` — it just adds `n_classes` on top of what `Model` already sets up.

Python also supports multiple inheritance — a class can inherit from more than one parent at once. When two parents define the same method, Python resolves which one wins using the Method Resolution Order (MRO): it walks the parents left to right as they were listed.

```
class Loggable:
    def info(self):
        return "[log] ready"

class Trainable:
    def info(self):
        return "[train] ready"

class Pipeline(Loggable, Trainable):
    pass

p = Pipeline()
print(p.info())
# [log] ready -- Loggable is listed first, so it wins
```

AI CONNECTION *scikit-learn builds many of its estimators this way: `RandomForestClassifier` inherits from both a base ensemble class and `ClassifierMixin`, which is exactly multiple inheritance — the mixin contributes shared scoring behavior without duplicating code.*

✓ QUICK CHECK — Inheritance & Multiple Inheritance

Q1. What does `super().__init__(name)` do inside a child class's constructor?

Answer: It calls the parent class's `__init__`, letting the child reuse the parent's setup instead of duplicating it.

Q2. If two parent classes define the same method, which one does a multiply-inheriting child use?

Answer: Python follows the Method Resolution Order (MRO), which by default checks the parents in the order they were listed, left to right.

05 Polymorphism & Encapsulation

Poly means "many" and morphism means "forms." Polymorphism means different classes can define a method with the same name, and calling that name on each object runs that object's own version.

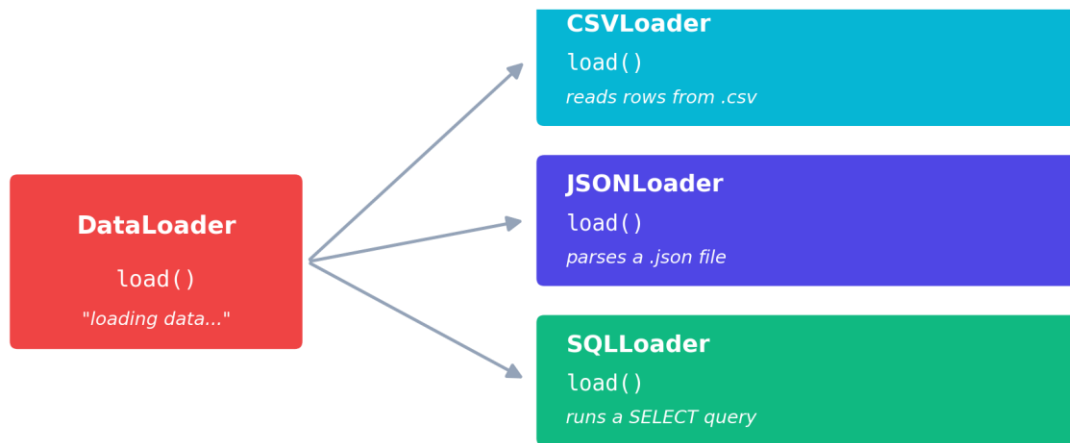


Figure 4.5 — Three loaders share the `load()` name but each implements it differently.

This is called method overriding — a subclass redefines a method it inherited:

```

class Model:
    def describe(self):
        print("A generic model")

class Classifier(Model):
    def describe(self):
        print("A classifier that predicts categories")

for m in [Model(), Classifier()]:
    m.describe()
# A generic model
# A classifier that predicts categories
  
```

Unlike some languages, Python does not support true method overloading — you cannot define the same method name twice with different parameter lists; the later definition simply replaces the earlier one. Python's usual workaround is default arguments or `*args`:

```
class Vector:
    def add(self, *numbers):
        return sum(numbers)

v = Vector()
print(v.add(1, 2))      # 3
print(v.add(1, 2, 3, 4)) # 10
```

Encapsulation means packing data and the functions that use it into one component, and controlling how that data can be reached from outside. A leading double underscore signals a private attribute Python will not let you access directly by name:

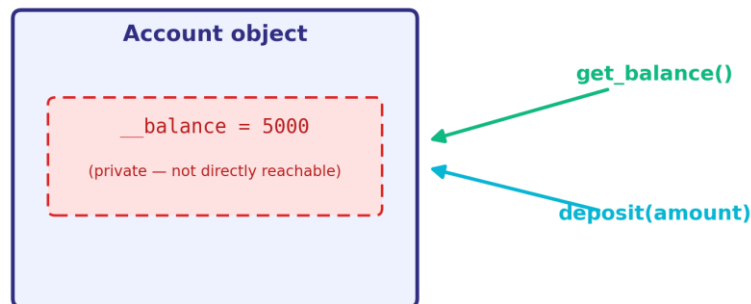


Figure 4.6 — Private data stays hidden; controlled methods are the only way in or out.

```
class Account:
    def __init__(self, balance):
        self.__balance = balance

    def get_balance(self):
        return self.__balance

acc = Account(5000)
print(acc.get_balance()) # 5000
print(acc.__balance)     # AttributeError
```

`@property` turns a method into something that reads like an attribute, and its matching `@x.setter` lets you validate a value before it is stored:

```
class Account:
    def __init__(self, balance):
        self.__balance = balance

    @property
    def balance(self):
        return self.__balance

    @balance.setter
    def balance(self, value):
        if value < 0:
            raise ValueError("balance can't be negative")
        self.__balance = value

acc = Account(5000)
acc.balance = -25
# ValueError: balance can't be negative
```

AI CONNECTION Every model's `.predict()` method is polymorphic — `RandomForestClassifier`, `LogisticRegression`, and an `SVM` all expose the same `predict(X)` name but each computes it differently. Encapsulation shows up as the private state inside a fitted scaler: you read `scaler.mean_` through a controlled attribute rather than reaching into its internals.

✓ QUICK CHECK — Polymorphism & Encapsulation

Q1. What does it mean for two classes to be polymorphic with respect to a method name?

Answer: Each class defines its own version of a method with the same name, and calling that name on an object runs that object's own implementation.

Q2. Does Python support classic method overloading (same name, different parameter lists)?

Answer: No — a later definition simply replaces the earlier one. Default arguments or `*args` are the usual Python workaround.

Q3. How do you make an attribute private, and how do you still expose it safely?

Answer: Prefix it with a double underscore (`self.__balance`), then expose controlled access through a method or an `@property` (optionally with a `@x.setter` that validates new values).

06 Special Methods & Functional Tips

Special methods (also called dunder methods, for their double underscores) let your objects plug into Python's built-in behavior — printing, length, and calling.

```
class Human:
    def __init__(self, name):
        self.name = name

    def __str__(self):
        return "Human: " + self.name

    def __len__(self):
        return len(self.name)

man = Human("Ahmed")
print(man)          # Human: Ahmed  (__str__)
print(len(man))     # 5              (__len__)
```

A handful of small functional tools come up constantly once you start writing classes: lambda for tiny anonymous functions, and map()/filter() for transforming or selecting items without writing a full loop.

```
square = lambda x: x * x
print(square(5)) # 25

scores = [55, 82, 91, 40, 76]
passed = list(filter(lambda s: s >= 60, scores))
doubled = list(map(lambda s: s * 2, scores))
print(passed) # [82, 91, 76]
print(doubled) # [110, 164, 182, 80, 152]
```

A generator produces values one at a time with yield instead of building the whole result in memory up front — handy when a dataset is too large to hold all at once:

```
def batches(data, size):
    for i in range(0, len(data), size):
        yield data[i:i + size]

for batch in batches([1, 2, 3, 4, 5], 2):
    print(batch)
# [1, 2]
# [3, 4]
# [5]
```

AI CONNECTION PyTorch's Dataset class expects you to implement `__len__` and `__getitem__` so the DataLoader knows how many samples exist and how to fetch one — the exact dunder-method pattern you just used. Generators are how most training loops stream mini-batches without loading an entire dataset into memory.

✓ QUICK CHECK — Special Methods & Functional Tips

Q1. What does defining `__str__` change about an object?

Answer: It controls what `print(obj)` or `str(obj)` shows, instead of the default `<__main__.ClassName object at 0x...>` text.

Q2. What is the key difference between a normal function that returns a list and a generator that yields values?

Answer: A generator produces values one at a time on demand and never holds the whole sequence in memory at once; a normal function builds and returns the entire list up front.

Practical Time

Work through these on your own before checking the marked answer.

Practice Question 1

What will this code print?

```
class Counter:
    count = 0
    def __init__(self):
        Counter.count += 1

Counter()
Counter()
print(Counter.count)
```

A. 0

B. 1

C. 2 ✓

D. AttributeError

Why: `Counter.count` is a class variable shared by every instance. Each `__init__` call increments the same shared value, so after two instantiations it reads 2.

Practice Question 2

What will this code print?

```
class Animal:
    def sound(self):
        return "..."
```

```
class Dog(Animal):
    def sound(self):
        return "Woof"
```

```
print(Dog().sound())
```

A. ...

B. Woof ✓

C. TypeError

D. None

***Why:** Dog overrides sound(), so calling it on a Dog instance runs Dog's version, not Animal's — this is method overriding, one form of polymorphism.*

Practice Question 3

Which of these correctly accesses a private attribute self.__score from outside the class?

A. obj.__score

B. obj._score

C. A public method or @property that returns it ✓

D. Private attributes cannot be read at all

***Why:** The double-underscore prefix triggers name mangling, so obj.__score raises an AttributeError from outside the class. The correct pattern is to expose it through a method or @property.*

Practice Question 4

What will this code print?

```
class Shape:
    def area(self):
        return 0

class Square(Shape):
    def __init__(self, side):
        self.side = side
    def area(self):
        return self.side ** 2

shapes = [Shape(), Square(4)]
print([s.area() for s in shapes])
```

- A. [0, 0]
- B. [0, 16] ✓**
- C. [16, 16]
- D. TypeError

*Why: Shape().area() uses the base implementation and returns 0. Square overrides area(), so Square(4).area() returns 4 ** 2 = 16.*

Practice Question 5

What does super().__init__(name) do inside a subclass's constructor?

- A. Creates a brand-new, unrelated object
- B. Calls the parent class's __init__ to reuse its setup logic ✓**
- C. Deletes the parent class
- D. Is only valid inside @staticmethod

Why: super() gives access to the parent class from within a subclass, letting the subclass reuse (rather than duplicate) the parent's constructor logic.

Practice Question 6

What will this code print?

```
class Box:
    @staticmethod
    def is_valid(size):
        return size > 0

print(Box.is_valid(-3))
```

- A. True
- B. False ✓**
- C. TypeError: missing self
- D. None

Why: is_valid is a static method — it takes no self/cls and can be called directly on the class. -3 > 0 is False, so that's what is returned and printed.

Practice Question 7

A function uses yield instead of return inside a loop. What kind of object does calling it produce?

- A. A list containing every yielded value
- B. A generator that produces values one at a time ✓**
- C. None — yield is invalid outside a class
- D. A dictionary

Why: yield turns a function into a generator function; calling it returns a generator object that lazily produces the next value each time it's iterated, rather than building the whole sequence up front.